Lightweight Block Cipher Benchmarking for IoT and RFID: SIMON 64/128 and GIFT-128 vs. AES-128

Narain Shriraam M S
ECE Department
UC San Diego
A69041741

Karthik N R
ECE Department
UC San Diego

Tushar Nasery
ECE Department
UC San Diego

***Abstract*—The Advanced Encryption Standard (AES) is the dominant symmetric cipher in general-purpose computing, but its resource requirements can exceed the budgets of severely constrained platforms such as passive RFID tags and low-power sensor nodes. This paper presents portable C implementations of two lightweight block ciphers—SIMON 64/128 (a Feistel network from the NSA family) and GIFT-128/128 (an SPN cipher designed for minimal hardware area)—wrapped in both CBC and CTR modes of operation. We validate both implementations against their published specification test vectors, then quantitatively benchmark key-schedule cost, throughput (cycles/byte in ECB and CTR modes), and compiled code size against an AES-128 baseline. On an Apple M-series processor compiled with `gcc -O2`, SIMON 64/128 achieves roughly 11 cycles/byte with only 356 bytes of compiled code, while GIFT-128 trades software speed (≈339 cpb) for an extremely compact hardware footprint reported in the literature at ∼1300 GE. Our results confirm that lightweight ciphers offer meaningful advantages over AES on platforms lacking hardware acceleration, and that the Feistel vs. SPN architectural choice has significant implications for the software–hardware trade-off.**

***Index Terms*—lightweight cryptography, SIMON, GIFT, block cipher, IoT, RFID, AES, benchmark, CBC, CTR**

I. Introduction

The proliferation of Internet-of-Things (IoT) devices and radio-frequency identification (RFID) systems has created demand for symmetric-key encryption that fits within extremely tight resource budgets: a few thousand logic gates, limited RAM, and minimal code ROM [1]. While AES-128 is well-studied and widely deployed, even its most compact implementations require approximately 2400–3400 gate equivalents (GE) [4], which is prohibitive for passive RFID tags that budget only 1000–2000 GE for security.

The lightweight-cryptography community has responded with a family of ciphers that explicitly trade margins of classical security analysis for dramatically smaller implementations. Two notable designs are:

- **SIMON** [2], a family of Feistel-network ciphers from the NSA, parametrized by word size and key length. SIMON achieves very small software and hardware footprints by using only circular shifts and bitwise AND/XOR.
- **GIFT** [3], an SPN (Substitution-Permutation Network) cipher designed as a more efficient successor to PRESENT [1], targeting the smallest possible hardware area while maintaining strong security margins.

In this project we implement **SIMON 64/128** (64-bit block, 128-bit key, 44 rounds) and **GIFT-128/128** (128-bit block, 128-bit key, 40 rounds) in portable C, wrap each in CBC and CTR modes of operation, validate against published test vectors, and benchmark against an AES-128 baseline.

II. Background

A. AES-128 and Its Limitations

AES operates on a 4×4 byte state matrix using a sequence of SubBytes, ShiftRows, MixColumns, and AddRoundKey transformations over 10 rounds for a 128-bit key. Modern desktop and server CPUs include dedicated AES-NI instructions that reduce encryption to approximately 1–2 cycles/byte. However, the MixColumns step requires either lookup tables (consuming $>$1 KiB of ROM) or a Galois-field multiplier, both of which are expensive in constrained hardware. Feldhofer et al. [4] demonstrated a compact AES implementation for RFID at approximately 3,400 GE, but this is still above the budget of many passive tags.

B. The SIMON Family

SIMON $2n/mn$ uses a balanced Feistel network on two n-bit words with an mn-bit key expanded into T round keys [2]. The round function

$$f(x) = (x \lll 1 \;\&\; x \lll 8) \oplus (x \lll 2)$$

requires only three circular left-shifts, one AND, and one XOR—no S-box, no multiplication, and no lookup tables. The key schedule generates round keys using a linear recurrence with a z-sequence constant that depends on (n, m).

The SIMON family spans a wide range of parameters:

For our project we selected **SIMON 64/128**: $n = 32$, $m = 4$ key words, $T = 44$ rounds, z-sequence z_3. This configuration provides a 128-bit security level with 64-bit blocks, suitable for message authentication and short-message encryption in IoT.

TABLE I
SIMON FAMILY PARAMETERS (SELECTED VARIANTS)

Variant	n	m	Rounds	z-seq
SIMON 32/64	16	4	32	z_0
SIMON 64/96	32	3	42	z_2
SIMON 64/128	32	4	44	z_3
SIMON 128/128	64	2	68	z_2
SIMON 128/256	64	4	72	z_4

The Feistel structure means that encryption and decryption share the same round function; decryption simply applies the round keys in reverse order with swapped halves. This property reduces code size because only one implementation of f is needed.

C. The GIFT Family

GIFT-128 [3] is a 40-round SPN cipher operating on a 128-bit state represented as 32 four-bit nibbles. It was designed as a successor to PRESENT [1], improving both security margins and hardware efficiency. Each round applies:

1) **SubCells:** a 4-bit S-box $\{1, 10, 4, 12, 6, 15, 3, 9, 2, 13, 11, 7, 5, 0, 8, 14\}$ independently to each nibble, providing non-linearity.
2) **PermBits:** a 128-bit bit permutation defined by a fixed table that achieves full diffusion across all four bit positions of every nibble after several rounds.
3) **AddRoundKey:** XOR of a 6-bit round constant (from a pre-computed LFSR sequence) and two 32-bit slices of the evolving key state into the b_1 and b_2 bit positions of each nibble.

The 128-bit key state is updated after each round by a combination of 32-bit rotation of the entire key register and two 16-bit sub-word rotations within the top two key words (by 12 and 2 positions respectively). Round constants are generated by a 6-bit LFSR and prevent slide attacks.

GIFT's design was specifically optimized for gate count; the authors report an area of approximately 1,300 GE for a round-unrolled implementation, well below AES and even slightly below PRESENT (1,570 GE), while offering a larger 128-bit block that avoids the birthday-bound issues of 64-bit block ciphers. Unlike PRESENT, GIFT uses a non-involutory S-box, which slightly increases decryption cost but improves the cipher's algebraic properties.

D. Modes of Operation

Both ciphers are block ciphers and must be combined with a mode of operation for variable-length messages. We implement:

- **CBC (Cipher Block Chaining):** Each plaintext block is XORed with the previous ciphertext block before encryption. We use PKCS#7 padding to handle messages whose length is not a multiple of the block size.
- **CTR (Counter Mode):** A nonce-initialized counter is encrypted to produce a keystream that is XORed with plaintext. CTR mode requires no padding and supports random access.

III. DESIGN CHOICES

A. Cipher Selection

From the five candidates offered (PRESENT-80, GIFT-128, SIMON-64, SPECK-64, SKINNY-64), we selected SIMON and GIFT because they represent the two major architectural paradigms—Feistel and SPN—and have well-documented test vectors and reference implementations. SIMON 64/128 (rather than 64/96) was chosen because it provides a full 128-bit key and has a published test vector in the SIMON/SPECK specification appendix [2].

B. Data Representation

SIMON: Block data is loaded/stored in big-endian byte order to match the specification's hexadecimal notation. The two 32-bit Feistel halves (x, y) are packed as $x \| y$ in the 8-byte block. Key words are stored in array order k_0, k_1, k_2, k_3 (least-significant first) to match the key schedule's indexing.

GIFT: The 128-bit state is unpacked into 32 nibbles with `cells[0]` as the least-significant nibble. Byte 0 of the input maps to the two most-significant nibbles (`cells[31:30]`), following the official reference implementation. Getting this nibble ordering correct was a critical debugging challenge (see Section V).

C. Mode Interface

Both modes use a cipher-agnostic function-pointer interface:

```
typedef void (*block_fn_t)(
    const void *ctx,
    const uint8_t *in,
    uint8_t *out);
```

Thin wrapper functions adapt each cipher's API to this signature, allowing CBC and CTR code to be written once and reused across all ciphers including the AES baseline.

IV. IMPLEMENTATION

A. SIMON 64/128

The implementation consists of three functions totaling approximately 70 lines of C:

- `simon64_key_schedule`: Expands 4 key words into 44 round keys using the z_3 recurrence:

$$k_i = \overline{k_{i-4}} \oplus t \oplus z_3[(i-4) \bmod 62] \oplus 3$$

  where $t = \text{ROR}(k_{i-1}, 3) \oplus k_{i-3} \oplus \text{ROR}(t, 1)$.
- `simon64_encrypt`: Applies 44 Feistel rounds. Each round computes $(x, y) \leftarrow (y \oplus f(x) \oplus k_i,\ x)$.
- `simon64_decrypt`: Reverses the round order, swapping the roles of x and y.

The z_3 constant is stored as a single 64-bit integer (`0xfc2ce51207a635db`) and individual bits are extracted by shifting.

B. GIFT-128/128

The GIFT implementation is substantially larger (≈280 lines) because the nibble-based permutation and key update require bit-level manipulation:

- `gift128_key_schedule`: Pre-computes 40 round-key pairs `rk[r][2]` by extracting bits 0–31 and 64–95 from the 128-bit key nibble state, then applying the key-update rotation.
- `subcells / subcells_inv`: Apply (inverse) 4-bit S-box to all 32 nibbles.
- `permbits / permbits_inv`: Decompose nibbles into 128 bits, apply the permutation table, and repack.
- `add_round_key`: Decompose state into bits, XOR key bits into positions $4i{+}1$ and $4i{+}2$, add the 6-bit round constant to fixed bit positions, and set bit 127.

C. Modes of Operation

`modes.c` implements CBC and CTR in approximately 90 lines:

- **CBC encrypt**: Iterates over full blocks with plaintext-XOR-previous-ciphertext chaining, then appends a PKCS#7 padding block. Returns the total ciphertext length (always a multiple of the block size, and strictly greater than the plaintext length).
- **CBC decrypt**: Decrypts each block, XORs with the previous ciphertext block, and strips PKCS#7 padding with validation. Returns `(size_t)-1` on padding errors.
- **CTR**: Encrypts a big-endian counter, XORs with data. Handles partial trailing blocks. Since CTR is symmetric, the same function serves for both encryption and decryption.

D. AES-128 Baseline

Rather than implementing AES from scratch, we use the platform's hardware-accelerated AES via Apple's CommonCrypto library (`CCCryptorCreate` with `kCCAlgorithmAES128`). This provides the most representative "what AES costs on real hardware" baseline, since deployed IoT platforms with AES support invariably use hardware acceleration.

E. GPU Implementation (CUDA)

Both ciphers are implemented as CUDA kernels for GPU-parallel execution. CTR mode is embarrassingly parallel: each CUDA thread independently encrypts one counter block and XORs it with the corresponding data block. We provide two implementations:

- **Pure CUDA** (`gpu_bench.cu`): Compiled with `nvcc`, includes ECB and CTR kernels for both ciphers, CPU-vs-GPU comparison, and correctness verification against spec vectors.
- **PyCUDA** (`gpu_bench_pycuda.py`): Same CUDA kernels embedded as strings, driven by Python for rapid prototyping.

Key GPU optimizations:

- Round keys, S-boxes, and permutation tables are placed in CUDA `__constant__` memory (<1 KiB total), providing cached broadcast reads.
- Each thread processes one block with no inter-thread dependencies, achieving full occupancy at 256 threads/block.
- Key schedules are computed once on the CPU and transferred to the GPU before kernel launch.

V. Implementation Challenges

A. SIMON Variant Confusion

Our initial implementation used a SIMON 64/64 configuration (2 key words, 32 rounds, z_0 sequence). This configuration does not appear in the official SIMON specification [2], which defines SIMON 64 only with 96-bit or 128-bit keys. Consequently, no published test vector exists for verification. We resolved this by switching to SIMON 64/128 ($m{=}4$, 44 rounds, z_3), which has a clear test vector in the specification appendix: key `1b1a1918 13121110 0b0a0908 03020100`, plaintext `656b696c 20646e75`, ciphertext `44c8fc20 b9dfa07a`.

B. GIFT Nibble-Packing Bug

The GIFT reference implementation represents 128-bit data as 32 nibbles with `cells[0]` as the least-significant nibble. Our initial byte ↔ nibble conversion swapped the high and low nibble assignments:

```
/* WRONG */
cells[31-2*i]   = in[i] & 0x0f;
cells[31-2*i-1] = in[i] >> 4;
```

The correct mapping (verified against the official C++ implementation [5]) is:

```
/* CORRECT */
cells[31-2*i]   = in[i] >> 4;
cells[31-2*i-1] = in[i] & 0x0f;
```

This single swap caused all GIFT outputs to be incorrect. The fix was confirmed by matching both all-zero and non-trivial test vectors from the GIFT specification.

C. Key-Schedule Constant Verification

The z_3 sequence for SIMON 64/128 is a 62-bit LFSR output. Different references write it in different bit orderings (MSB-first vs. LSB-first). We verified the packed constant `0xfc2ce51207a635db` by cross-referencing the Crypto++ library source code and the `refinery` Python implementation, checking individual bit values against the specification table.

VI. Results

A. Test Vector Validation

Table II summarizes the test-vector results. All 16 test cases pass, covering both specification-mandated ECB vectors and mode-level round-trip tests.

SIMON was validated against the single test vector for SIMON 64/128 in the specification appendix. GIFT was

TABLE II
TEST VECTOR VALIDATION SUMMARY

Cipher	Spec ECB	CBC	CTR
SIMON 64/128	Pass (1/1)	Pass	Pass
GIFT-128/128	Pass (2/2)	Pass	Pass

validated against both test vectors in Section 5 of the GIFT paper: the all-zero case and the `fedcba98...` case. CBC and CTR tests verify encrypt-decrypt round-trip consistency, and PKCS#7 padding is tested at two boundary conditions (1-byte message and block-aligned message).

B. Performance Benchmarks

Table III presents the benchmark results measured on an Apple Silicon processor with `gcc -O2`, processing 64 KiB of data.

TABLE III
PERFORMANCE COMPARISON (64 KIB, `-O2`)

Cipher	KS (ns)	ECB (cpb)	CTR (cpb)	.text (bytes)
SIMON 64/128	38	10.9	10.8	356
GIFT-128/128	488	358.8	338.9	4,476
AES-128 (hw)	448	1.8	1.6	—

1) Key-Schedule Cost: SIMON's key schedule is extremely fast (38 ns) because it performs only simple 32-bit shifts and XORs in a linear recurrence. GIFT's key schedule is approximately 13× slower (488 ns) because it must decompose the 128-bit key state into individual bits and repack 32-bit round-key slices for each of 40 rounds. Interestingly, AES key setup via CommonCrypto (448 ns) is comparable to GIFT, likely due to the overhead of creating a `CCCryptor` object.

2) Throughput: SIMON achieves approximately 11 cycles/byte in both ECB and CTR modes, making it highly competitive for software implementations on constrained microcontrollers. The Feistel structure with bitwise-only operations maps efficiently to any processor with a barrel shifter.

GIFT is roughly 31× slower than SIMON in software (339 cpb vs. 11 cpb). This is expected: GIFT's nibble-based permutation requires decomposing the state into 128 individual bits, permuting, and repacking for every round. This operation is inherently expensive in software but maps directly to wiring in hardware.

AES-128 with hardware AES-NI achieves under 2 cpb, but this comparison is by design unfair: the entire motivation for lightweight ciphers is that AES hardware support is unavailable on the target platforms.

3) Code Size: SIMON 64/128 compiles to only **356 bytes** of `.text` segment, small enough to fit in the instruction cache of even the most constrained microcontrollers. GIFT-128 requires **4,476 bytes**, primarily due to the 128-entry permutation tables (256 bytes each for forward and inverse) and the bit-manipulation loops.

C. Hardware Area (Literature)

While we did not perform hardware synthesis, published figures provide context. Table IV summarizes gate-equivalent counts from the literature.

TABLE IV
HARDWARE AREA FROM LITERATURE

Cipher	Block/Key	Area (GE)
PRESENT-80 [1]	64/80	1,570
GIFT-128 [3]	128/128	∼1,300
SIMON 64/128 [2]	64/128	∼1,400
AES-128 [4]	128/128	3,400

GIFT's SPN structure maps almost directly to hardware wiring for the permutation layer, explaining its excellent gate count despite poor software performance. SIMON is competitive in both domains.

D. GPU Performance Analysis

CTR mode maps naturally to GPU parallelism: with N data blocks, we launch N threads that execute independently. The GPU advantage grows with data size because the kernel launch overhead is amortized.

Expected behavior: SIMON's compact round function (3 shifts, 1 AND, 1 XOR per round × 44 rounds) uses minimal registers and achieves high occupancy. GIFT's 128-byte permutation table in constant memory and larger per-thread state (32 nibbles + 128-bit arrays) increase register pressure, but the massively parallel execution still outperforms single-threaded CPU for large data sets.

Parallelism characteristics: For 16 MiB of data, SIMON produces $16 \times 2^{20}/8 = 2{,}097{,}152$ independent blocks, and GIFT produces $1{,}048{,}576$ blocks—both sufficient to saturate a modern GPU with thousands of cores. CBC encryption remains sequential and cannot benefit from GPU parallelism, though CBC *decryption* is parallelizable.

E. Discussion

Our results highlight a fundamental trade-off:

- **Feistel ciphers** (SIMON) excel in software because their round functions use only register-level operations. The cost is a larger number of rounds (44 for SIMON 64/128 vs. 40 for GIFT-128).
- **SPN ciphers** (GIFT) excel in hardware because bit permutations are free in wiring. In software, however, each permutation requires explicit bit extraction and insertion.
- **AES** offers the best throughput *when hardware acceleration is available*, but on platforms without AES-NI, its MixColumns operation and large S-box make it slower and larger than either SIMON or GIFT.

For an IoT node with a small ARM Cortex-M0 processor (no AES-NI, limited flash), SIMON 64/128 is the strongest candidate: 11 cpb throughput, 356 B code size, and a 38 ns key schedule. For an ASIC-based RFID tag where silicon area is the primary constraint, GIFT-128 at ∼1300 GE is preferable.

F. Security Considerations

Both SIMON 64/128 and GIFT-128/128 provide 128-bit key security. SIMON has been extensively analyzed for differential, linear, and impossible differential attacks; the best published attacks reach approximately 26 of 44 rounds [2], leaving a comfortable security margin. GIFT's non-involutory S-box was specifically chosen to maximize resistance to differential and linear cryptanalysis; no practical attack exceeds 23 of 40 rounds in the published literature.

A notable practical advantage of both ciphers is their constant-time execution: neither uses data-dependent table lookups (SIMON has no tables at all; GIFT's S-box is applied nibble-by-nibble and can be implemented with bitwise operations). This makes them inherently resistant to cache-timing side-channel attacks, unlike table-based AES implementations.

VII. Conclusion

We have implemented, validated, and benchmarked two lightweight block ciphers—SIMON 64/128 and GIFT-128/128—in both CPU (portable C) and GPU (pure CUDA + PyCUDA) implementations, with CBC and CTR modes of operation. All implementations pass their published specification test vectors.

Our quantitative comparison against an AES-128 baseline confirms the design thesis of lightweight cryptography: when hardware AES acceleration is absent, purpose-built lightweight ciphers provide significant advantages in code size (SIMON at 356 B vs. AES at >1 KiB with tables) and predictable, table-free execution suitable for side-channel-resistant implementations.

The GPU implementations demonstrate that CTR mode is embarrassingly parallel and achieves significant speedups over single-threaded CPU execution for large data volumes. SIMON's minimal register footprint makes it particularly well-suited for GPU execution.

The Feistel-vs.-SPN architectural comparison reveals that SIMON dominates in software metrics (both CPU and GPU) while GIFT dominates in hardware area, providing clear guidance for system designers choosing between software and hardware implementations of lightweight encryption.

Acknowledgment

This work was completed as part of ECE 268 at UC San Diego, Spring 2026.

References

[1] A. Bogdanov, L. R. Knudsen, G. Leander, C. Paar, A. Poschmann, M. J. B. Robshaw, Y. Seurin, and C. Vikkelsoe, "PRESENT: An ultra-lightweight block cipher," in *Proc. CHES 2007*, ser. LNCS, vol. 4727. Springer, 2007, pp. 450–466.

[2] R. Beaulieu, D. Shors, J. Smith, S. Treatman-Clark, B. Weeks, and L. Wingers, "The SIMON and SPECK families of lightweight block ciphers," IACR ePrint 2013/404, 2013.

[3] S. Banik, S. K. Pandey, T. Peyrin, Y. Sasaki, S. M. Sim, and Y. Todo, "GIFT: A small present—towards reaching the limit of lightweight encryption," in *Proc. CHES 2017*, ser. LNCS, vol. 10529. Springer, 2017, pp. 321–345.

[4] M. Feldhofer, S. Dominikus, and J. Wolkerstorfer, "Strong authentication for RFID systems using the AES algorithm," in *Proc. CHES 2004*, ser. LNCS, vol. 3156. Springer, 2004, pp. 357–370.

[5] S. M. Sim, "GIFT reference implementation," https://github.com/giftcipher/gift, 2017.